

Rapport d'Architecture Logicielle

Projet de Fin d'Année : Plateforme Éducative & GMAO

Équipe de Développement

8 mai 2026

Table des matières

1	Introduction et Vision du Projet	2
2	Stack Technologique	2
3	Architecture Logicielle & Design Patterns	2
3.1	L'Architecture MVT (Model-View-Template)	2
3.2	Design Pattern : RBAC (Role-Based Access Control)	2
3.3	Pattern de Rendu Partiel (Partial Rendering avec HTMX)	2
4	Structure des Modules (Applications Django)	3
4.1	Module 1 : Accounts (Gestion des Utilisateurs)	3
4.2	Module 2 : Apprentissage (LMS)	3
4.3	Module 3 : Logistics (GMAO / Inventaire)	3
5	Problèmes Techniques Rencontrés et Résolutions	3
5.1	Affichage des Documents PDF (Erreur 153 & Refused to Connect)	3
5.2	Intégration Asynchrone (HTMX CSRF Token)	4
5.3	Gestion du Routage Dynamique (ModuleNotFoundError)	4
6	Interfaces et Expérience Utilisateur (UX)	4
7	Conclusion et Perspectives (Phase 5)	4

1 Introduction et Vision du Projet

Le présent document détaille l'architecture logicielle, les choix technologiques et les modèles de conception appliqués lors du développement de la **Plateforme Éducative IoT et IA**. Ce projet hybride intègre un système de gestion de l'apprentissage (LMS) et un module de Gestion de Maintenance Assistée par Ordinateur (GMAO) pour la logistique du matériel physique (capteurs IoT, Raspberry Pi, etc.).

2 Stack Technologique

Le projet repose sur une pile technologique robuste et moderne, favorisant le développement rapide (RAD) et la maintenabilité :

- **Backend Framework** : Django (Python). Choisi pour son ORM puissant, son architecture modulaire et sa sécurité intégrée.
- **Frontend Hypermedia** : HTMX. Utilisé pour dynamiser l'interface (Single Page Application feel) sans la complexité d'un framework JavaScript lourd (comme React ou Angular).
- **Base de données** : Modèle relationnel (SQL) géré par l'ORM Django.
- **Styling** : CSS personnalisé (orienté composants via `app.css`).

3 Architecture Logicielle & Design Patterns

3.1 L'Architecture MVT (Model-View-Template)

Le projet suit strictement l'architecture MVT de Django :

- **Models** : Définition des structures de données en Python (classes) mappées sur la base de données.
- **Views** : Logique métier. Réception des requêtes HTTP, interrogation des modèles et formatage des données.
- **Templates** : Présentation HTML dynamique utilisant le moteur de template Django.

3.2 Design Pattern : RBAC (Role-Based Access Control)

Un système de contrôle d'accès basé sur les rôles a été implémenté pour séparer les logiques :

- **Étudiant** : Accès en lecture au catalogue de cours.
- **Formateur** (`is_formateur=True`) : Accès exclusif à l'"Espace Formateur" pour créer et modifier *ses propres* cours. Utilisation de permissions strictes via des vérifications au niveau des Vues.

3.3 Pattern de Rendu Partiel (Partial Rendering avec HTMX)

Pour moderniser l'UX sans créer d'API REST complète, le projet utilise le rendu de fragments HTML :

```
<!-- Bouton declenchant la requete asynchrone -->
<button hx-get="/chapitre/ajouter/" hx-target="#form-container">
  Ajouter
</button>
```

Le backend Django intercepte la requête et renvoie uniquement un fragment HTML (`chapitre_form.html`), réduisant considérablement la charge réseau.

4 Structure des Modules (Applications Django)

L'architecture est modulaire, divisée en applications indépendantes pour respecter le principe de responsabilité unique (Single Responsibility Principle).

4.1 Module 1 : Accounts (Gestion des Utilisateurs)

Extension du modèle utilisateur natif de Django pour intégrer des champs personnalisés spécifiques au monde académique (ex: `is_formateur`).

4.2 Module 2 : Apprentissage (LMS)

Ce module gère le contenu pédagogique.

Schéma Relationnel Simplifié (Apprentissage) :

```
[ Utilisateur (Formateur) ] 1 ----- * [ Cours ]
[ Cours ] 1 ----- * [ Chapitre ]
[ Chapitre ] 1 ----- * [ Document (PDF/Video) ]
```

4.3 Module 3 : Logistics (GMAO / Inventaire)

Ce module gère le matériel physique utilisé lors des ateliers.

Schéma Relationnel Simplifié (Logistique) :

```
[ Equipment ] (Disponible, En Panne, Maintenance)
[ Workshop (Atelier) ] * ----- 1 [ Utilisateur (Tuteur) ]
[ Ticket ] * ----- 1 [ Equipment ] (Optionnel)
[ Ticket ] * ----- 1 [ Workshop ] (Optionnel)
[ Ticket ] * ----- 1 [ Utilisateur (Createur) ]
```

5 Problèmes Techniques Rencontrés et Résolutions

Au cours du développement, plusieurs défis architecturaux et de sécurité ont été surmontés :

5.1 Affichage des Documents PDF (Erreur 153 & Refused to Connect)

Problème : Les navigateurs modernes bloquent le rendu des PDF locaux via les balises `<object>` ou les iframes en raison des politiques de sécurité "Cross-Origin" ou "Clickjacking".

Solution :

- Remplacement de la balise `<object>` par une `<iframe>`.

- Configuration du Middleware Django : Ajout de `X_FRAME_OPTIONS = 'SAMEORIGIN'` dans `settings.py` pour autoriser l'affichage en iframe depuis le même domaine.

5.2 Intégration Asynchrone (HTMX CSRF Token)

Problème : Les requêtes POST envoyées par HTMX pour modifier des données (ex: résoudre un ticket, ajouter un chapitre) étaient bloquées par Django (Erreur 403 Forbidden).

Solution : Configuration globale de HTMX dans le template de base pour inclure automatiquement le jeton CSRF de Django dans les en-têtes HTTP de chaque requête asynchrone :

```
<script>
  document.body.addEventListener('htmx:configRequest', (event) => {
    event.detail.headers['X-CSRFToken'] = '{{ csrf_token }}';
  });
</script>
```

5.3 Gestion du Routage Dynamique (ModuleNotFoundError)

Problème : Déclaration d'une route d'application dans le fichier principal `urls.py` avant l'existence physique du sous-fichier de routage, entraînant un crash du serveur de développement.

Solution : Adoption d'une méthodologie de développement incrémentale "Bottom-Up" : création des vues, puis des urls locales, puis enregistrement dans le routeur global.

6 Interfaces et Expérience Utilisateur (UX)

Le projet se détache délibérément de l'interface d'administration native de Django (Django Admin) en proposant un **Espace Formateur** dédié. Ce choix d'architecture front-end permet :

- De masquer la complexité de la base de données aux utilisateurs finaux.
- D'intégrer des flux métier spécifiques (ex: Création d'un cours → Redirection automatique → Ajout de chapitres en asynchrone).
- De restreindre les champs modifiables (ex: lors de l'ouverture d'un Ticket logistique, le statut initial est forcé à "OUVERT" côté serveur, sans que l'utilisateur n'ait à s'en soucier).

7 Conclusion et Perspectives (Phase 5)

L'architecture actuelle forme un socle solide pour la phase finale du PFA : l'intégration de l'Intelligence Artificielle et de l'Internet des Objets (IoT). Grâce à la séparation claire des applications (Apprentissage vs Logistique), le système est prêt à accueillir des agents IA (ex: pour l'aide à la création de cours) ou à recevoir des données télémétriques automatisées provenant des équipements (IoT) pour générer des tickets de maintenance automatiquement.